KRISTU JAYANTI
(DEEMED TO BE UNIVERSITY)
Under Section 3 of UGC Act 1956
A CMI INSTITUTION | BENGALURU | INDIA

COURSE PROGRESS MANAGEMENT SYSTEM

Project Report submitted in partial fulfilment of the requirements for the Award of the degree of

BACHELOR OF COMPUTER APPLICATIONS (BCA)

Department of Computer Science (UG)

Submitted by

KUMARAN S
24BCAC22

Under the guidance of

Ms. Mohana Priya S

DEPARTMENT OF COMPUTER SCIENCE (UG)

BCA PROGRAMME

KRISTU JAYANTI (Deemed to be University)

K. Narayanapura, Kothanur P.O., Bangalore – 5600774

KRISTU JAYANTI
(DEEMED TO BE UNIVERSITY)
Under Section 3 of UGC Act 1956
A CMI INSTITUTION | BENGALURU | INDIA

DEPARTMENT OF COMPUTER SCIENCE (UG)

CERTIFICATE OF COMPLETION

This is to certify that the project entitled "**COURSE PROGRESS MANAGEMENT SYSTEM"** has been satisfactorily completed by **KUMARAN S**, **24BCAC22** in partial fulfilment of the award of the Bachelor of Computer Applications degree requirements prescribed by Kristu Jayanti (Deemed to be University) Bengaluru during the academic year 2025-26.

Internal Guide ***Head of the Department***

Valued by Examiners

1: _____ ***Centre:*** **Kristu Jayanti Univesity**

2: _____ ***Date:***

KRISTU JAYANTI
(DEEMED TO BE UNIVERSITY)
Under Section 3 of UGC Act 1956
A CMI INSTITUTION | BENGALURU | INDIA

DECLARATION

I, KUMARAN S, 24BCAC22 hereby declare that the project work entitled "COURSE PROGESS MANAGEMENT SYSTEM" is an original project work carried out by me, under the guidance of Ms. Mohana Priya S

This project work has not been submitted earlier either to any University / Institution or any other body for the fulfillment of the requirement of a course of study.

Signature
KUMARAN S

Bengaluru
Date:

ACKNOWLEDGEMENT

The success of the project depends upon the efforts invested. It's my duty to acknowledge and thank the individuals who has contributed to the successful completion of the project.

I take this opportunity to express my profound and wholehearted thanks to Rev. Fr. Dr. **AUGUSTINE GEORGE, PRINCIPAL & VICE CHANCELLOR, AND Rev. Fr. LIJO P. THOMAS, PRO VICE CHANCELLOR IN KRITSU JAYANTI (Deemed to be University),**, BANGALORE for providing ample facilities made to undergo my project successfully.

I express my deep sense of gratitude and sincere thanks to our **DEAN Dr. R. KUMAR** for his valuable support.

I express my deep sense of gratitude and sincere thanks to our Head of the Department **Dr. SEVUGA PANDIAN** and Programme Coordinator **Dr. RANJITHA M** for their valuable advice.

I feel immense pleasure to thank my respected guide Ms. Mohana Priya S for sustaining interest and providing dynamic guidance in aiding me to complete this project immaculately and impeccably and for being the source of my strength and confidence.

It is my duty to express my thanks to all Teaching and Non-Teaching Staff members of Computer Science department who offered me help directly or indirectly by their suggestions. The successful completion of my project would not have been possible without my parent's sacrifice, guidance, and prayers. I take this opportunity to thank everyone for their continuous encouragement. I convey my thankfulness to all my friends who were with me to share my happiness and agony.

Last but not the least I thank almighty God for giving me strength and good health throughout my project and enabling me to complete it successfully.

SYNOPSIS

The Student Course Management System is a web-based application designed to manage interaction between students and faculty in an academic environment. The system provides a centralized platform where faculty can create courses and students can enroll in them. It simplifies course management, improves accessibility, and ensures proper tracking of student progress.

The system consists of two main users: Student and Faculty. Students can view available courses, enroll in them, and track their enrolled courses. Faculty members can add courses, manage them, and view the list of students enrolled in each course along with their progress. The application uses a frontend (HTML, CSS, JavaScript) and a backend database (Supabase/PostgreSQL) to store and retrieve data in real time.

TABLE OF CONTENTS

INTRODUCTION

1.INTRODUCTION

1.1 PROBLEM DEFINITION

The Student Course Management System is a web-based application. The main purpose of the "Student Course Management System" is to provide a convenient way for students to view and enroll in courses offered by faculty members. The objective of this project is to develop a system that automates the processes and activities of course management in an academic environment. In this project, we will make it easier for students to browse available courses and for faculty to manage and track student enrollment.

1.2 SCOPE OF THE PROJECT

In the present system, students and faculty have to rely on manual methods or scattered digital tools to manage course enrollment and progress tracking. This often requires a lot of time and effort. It is tedious for students to track their enrolled courses and for faculty to monitor student progress. The project 'Student Course Management System' is developed to replace the currently existing manual system, which helps in keeping centralized records of courses, enrollments, and student progress.

Modules in the software

- Student Module
- Login, Registration, View Profile, Update Information.

- Administrator / Faculty Module

This module provides faculty-related functionality like managing course information, adding course details, managing student enrollments, and viewing student progress. From this module, faculty can add, delete, edit, and view courses and enrolled students.

1. Manage student information.
2. Update course information.

3. Manage course offerings.
4. View enrolled students per course.
5. Track student progress.

- Student Module

Students can view available courses, enroll in courses of their choice, and track the status of their enrolled courses. Students can also view their course history and progress details.

6. Registration (as Student)
7. Login and Dashboard
8. Browse and Search Courses
9. Enroll in Courses
10. View My Courses and Progress

- Course Management Module

Faculty members can create new courses, update existing course details, and manage course content. Each course is linked to a faculty member and can have multiple student enrollments.

- Enrollment Module

This module manages the enrollment of students in courses. Students can enroll in available courses, and faculty can view and manage their enrolled student lists. The module maintains details of all enrollments made and allows tracking of status (Pending / In Progress / Completed).

- Progress Tracking Module

This module tracks the enrollment status and progress of each student in their enrolled courses. The status can be Pending, In Progress, or Completed, allowing both students and faculty to monitor academic progress.

SYSTEM STUDY

2.SYSTEM STUDY

2.1EXISTING SYSTEM

In the present system, students and faculty have to manage course information manually or through disconnected tools. Students have no centralized platform to view available courses, and faculty have no easy way to monitor student progress. This often requires a lot of time and effort. It is tedious for both students and faculty to maintain and update records of enrollments and progress.

Disadvantages of the Existing System

- Difficulty in tracking course enrollment
- Lack of progress monitoring for students
- No centralized management of course data
- Manual and time-consuming processes
- Unnecessary navigation across multiple disconnected platforms

2.2FEASIBILITY STUDY

A feasibility study is an analysis of how successfully a project can be completed, accounting for factors that affect it such as economic, technological, legal and scheduling factors. Project managers use feasibility studies to determine potential positive and negative outcomes of a project before investing a considerable amount of time and money into it. A feasibility study tests the viability of an idea, a project or even a new business. The goal of a feasibility study is to place emphasis on potential problems that could occur if a project is pursued and determines if, after all significant factors are considered, the project should be pursued. Feasibility studies also allow a business to address where and how it will operate, potential obstacles, competition and the funding needed to get the business up and running.

This project "Student Course Management System" has undergone the following Feasibility study:

- Economic Feasibility

- Technical Feasibility
- Behavioural Feasibility
- Schedule Feasibility

Every project is feasible for given unlimited resources and infinite time. Feasibility study is an evaluation of the proposed system regarding its workability, impact on the organization, ability to meet the user needs and effective use of resources. Thus, when a new application is proposed it normally goes through a feasibility study before it is approved for development. Feasibility and risk analysis are related in many ways. The feasibility analysis in this project has been discussed below based on the above-mentioned components of feasibility.

1. Technical Feasibility:

Technical feasibility focuses on the technology used. It means the computerized system is technically feasible, i.e., it doesn't have any technical fault and works properly in the given environment. The system is technically feasible if it provides the required output. The Student Course Management System uses well-established web technologies (HTML, CSS, JavaScript) and a reliable backend (Supabase/PostgreSQL). The system does not require any specialized hardware or software beyond a standard web browser, making it highly accessible and technically sound.

2. Economic Feasibility:

Economic analysis is the most frequently used method for evaluating the effectiveness of a computerized system. The Student Course Management System is economically feasible because it uses open-source and free-tier tools like Supabase and standard web technologies (HTML, CSS, JavaScript). The cost of development and deployment is minimal compared to a manual system. It saves time, reduces manpower requirements, and is cost-effective according to cost-benefit analysis.

3. Behavioural Feasibility:

Behavioural feasibility is the analysis of user acceptance of the system. In this, we analyze that the computerized system is working properly and communicating properly with the environment. The application is designed with a clean, user-friendly interface so that both students and faculty can easily navigate and use the system without extensive training. All the matters are analyzed and a good computerized system is prepared to ensure smooth behavioural feasibility.

4. Schedule Feasibility:

Time evaluation is the most important consideration in the development of the project. The time schedule required for the development of the project is very important since more development time affects machine time, cost and causes delay in the development of other systems. The Student Course Management System has been carefully planned with clearly defined modules and tasks, making it feasible to complete within the given academic timeline.

2.3PROPOSED SYSTEM

The Student Course Management System is a web-based application designed to provide a convenient and centralized platform for students and faculty. The main purpose of this system is to allow faculty to create and manage courses while enabling students to view, enroll in, and track their course progress in real time.

The proposed system provides the following advantages:

- Centralized course and enrollment management
- Real-time data storage and retrieval using Supabase/PostgreSQL
- Easy and accessible interface for both students and faculty
- Automated tracking of student enrollment and progress
- Reduces manual effort and saves time
- Accessible from any device with a web browser

SYSTEM DESIGN

3.SYSTEM DESIGN

In the design phase, the architecture of the system is established. This phase starts with the requirement document delivered by the requirement phase and maps the requirements into architecture. The architecture defines the components, their interfaces and behaviours. The deliverable design document is the architecture.

The design document describes a plan to implement the requirements. This phase represents the 'how' phase. Details on computer programming languages and environments, machines, packages, application architecture, distributed architecture layering, memory size, platform, algorithms, data structures, global type definitions, interfaces, and many other engineering details are established.

System design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. Systems design could be seen as the application of systems theory to product development. There is some overlap with the disciplines of systems analysis, systems architecture and systems engineering.

Object-oriented analysis and design methods are becoming the most widely used methods for computer systems design. The UML has become the standard language in object-oriented analysis and design. It is widely used for modelling software systems and is increasingly used for designing non-software systems and organizations.

LOGICAL DESIGN

The logical design of a system pertains to an abstract representation of the data flows, inputs and outputs of the system. This is often conducted via modelling, using an abstract (and sometimes graphical) model of the actual system. In the context of the Student Course Management System, logical design includes entity-relationship diagrams (ER diagrams) and Data Flow Diagrams (DFDs).

PHYSICAL DESIGN

The physical design relates to the actual input and output processes of the system. This is explained in terms of how data is input into a system, how it is verified/authenticated, how it is processed, and how it is displayed.

In physical design, the following requirements about the system are decided:

11. Input requirement,
12. Output requirements,
13. Storage requirements,
14. Processing requirements,
15. System control and backup or recovery.

Put another way, the physical portion of system design can generally be broken down into three sub-tasks:

16. User Interface Design
17. Data Design
18. Process Design

User Interface Design is concerned with how users add information to the system and with how the system presents information back to them. Data Design is concerned with how the data is represented and stored within the system. Finally, Process Design is concerned with how data moves through the system, and with how and where it is validated, secured and/or transformed as it flows into, through and out of the system.

E-R DIAGRAM

3.1 E-R DIAGRAM

An entity relationship diagram (ERD) shows the relationships of entity sets stored in a database. An entity in this context is a component of data. In other words, ER diagrams illustrate the logical structure of databases.

Structure of an Entity Relationship Diagram with Common ERD Notations

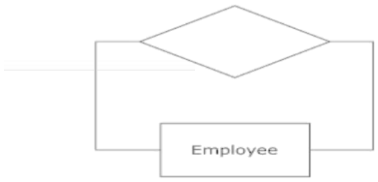
An entity relationship diagram is a means of visualizing how the information a system produces is related. There are five main components of an ERD:

- Entities, which are represented by rectangles. An entity is an object or concept about which you want to store information. In the Student Course Management System, the main entities are: Student, Faculty, Course, and Enrollment.

- Weak Entity is an entity that must be defined by a foreign key relationship with another entity as it cannot be uniquely identified by its own attributes alone.

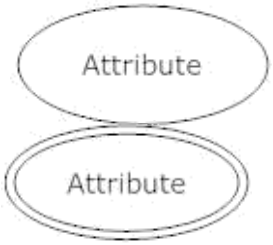
- Actions / Relationships, which are represented by diamond shapes, show how two entities share information in the database. In the Student Course Management System: Faculty CREATES Course; Student ENROLLS IN Course (via Enrollment entity).

•



- Relationship: The degree of a relationship is the number of entity types that participate in the relationship. Types of relationships in this system:
  - One Faculty creates Many Courses (One to Many)
  - One Student can enroll in Many Courses (Many to Many via Enrollment)

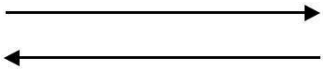
- Attributes, which are represented by ovals. A key attribute is the unique, distinguishing characteristic of the entity. For example, student_id is the primary key attribute of the Student entity; course_id is the primary key of the Course entity.



- Multi-valued attribute can have more than one value. For example, a student entity can have multiple enrollment values (multiple course enrollments).

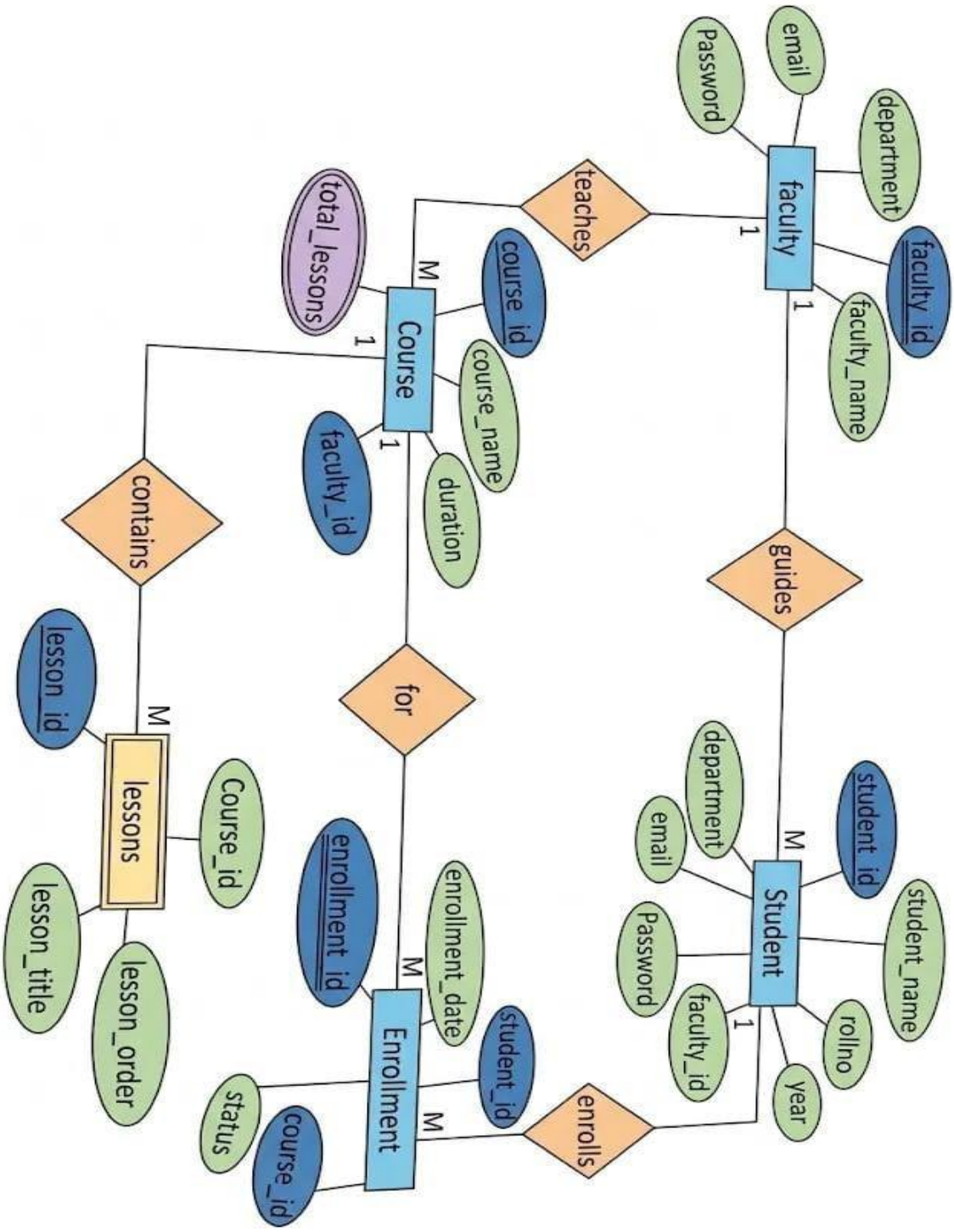
- Derived attribute is based on another attribute. For example, the enrollment count of a faculty member is derived from the number of students enrolled in their courses.

- Connecting lines, solid lines that connect attributes to show the relationships of entities in the diagram.



- Cardinality specifies how many instances of an entity relate to one instance of another entity. Cardinality types in this system:
  - One to One
  - One to Many
  - Many to One
  - Many to Many

E-R DIAGRAM FOR STUDENT COURSE MANAGEMENT SYSTEM:



3.2 DATA FLOW DIAGRAM (DFD) [Level 0 and Level 1]

The Data Flow Diagrams (DFDs) are used for structure analysis and design. DFDs show the flow of data from external entities into the system. DFDs also show how the data moves and are transformed from one process to another, as well as its logical storage. The following symbols are used within DFDs.

A data flow diagram (DFD) is a graphical representation of the "flow" of data through an information system, modelling its process aspects. A DFD is often used as a preliminary step to create an overview of the system, which can later be elaborated. DFDs can also be used for the visualization of data processing (structured design).

A DFD shows what kind of information will be input to and output from the system, where the data will come from and go to, and where the data will be stored. It does not show information about the timing of processes or information about whether processes will operate in sequence or in parallel.

3.2PHYSICAL VS LOGICAL DFD

A logical DFD captures the data flows that are necessary for a system to operate. It describes the processes that are undertaken, the data required and produced by each process, and the stores needed to hold the data. On the other hand, a physical DFD shows how the system is actually implemented, either at the moment (Current Physical DFD), or how the designer intends it to be in the future (Required Physical DFD).

A Logical DFD attempts to capture the data flow aspects of a system in a form that has neither redundancy nor duplication, while a Physical DFD shows the actual implementation details and data stores as they exist or will exist.

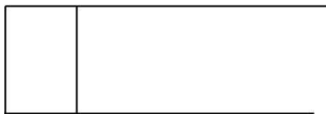
DATA FLOW SYMBOLS AND THEIR MEANINGS:

An Entity (External Entity / Source/Sink):

A source of data or a destination for data. Represented by rectangles in the diagram. Sources and Sinks are external entities which are sources or destinations of data. In the Student Course Management System, external entities are: Student, Faculty.

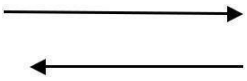
Process:

Represented by circles in the diagram. Processes are responsible for manipulating the data. They take data as input and output an altered version of the data. Examples: Login Process, Course Management Process, Enrollment Process.



Data Store:

Represented by a segmented rectangle with an open end on the right. Data Stores are both electronic and physical locations of data. In this system, data stores include: Student Table, Faculty Table, Course Table, Enrollment Table (all in Supabase/PostgreSQL database).



Data Flow:

Represented by a unidirectional arrow. Data Flows show how data is moved through the system. Data Flows are labelled with a description of the data being passed through them.

Level 0 DFD / Context Diagram:

A level-0 DFD is the most basic form of DFD. It aims to show how the entire system works at a glance. There is only one process in the system and all the data flows either into or out of this process. Level-0 DFD demonstrates the interactions between the process and external entities.

Level 1 DFD:

Level 1 DFD aims to give an overview of the full system. It looks at the system in more detail. Major processes are broken down into sub-processes. Level 1 DFD also identifies data stores that are used by the major processes.

Working flow of the Student Course Management System:

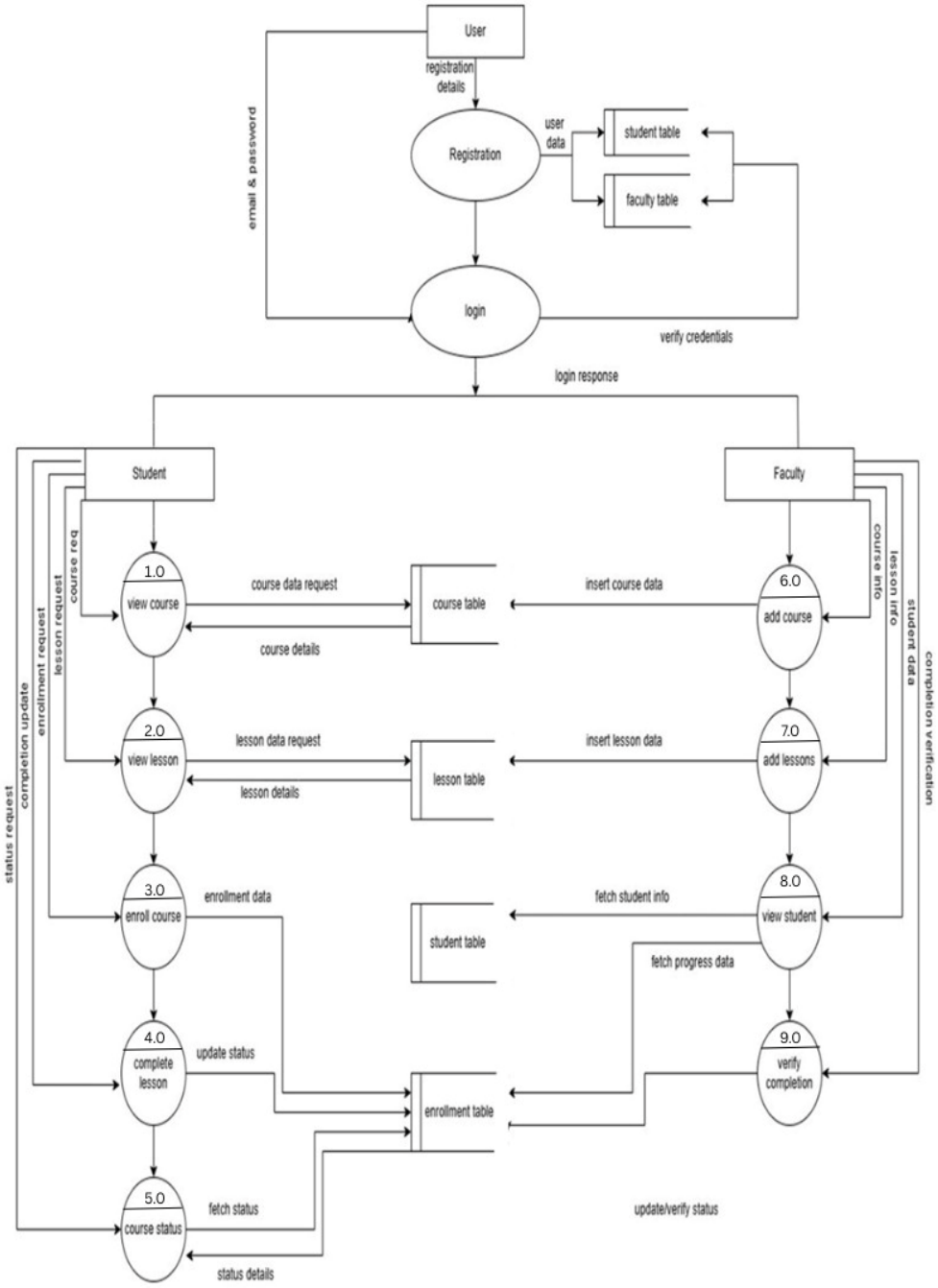
Student Flow:

Login → View Available Courses → Click Enroll → Course added to student profile → View in My Courses → Track Progress Status

Faculty Flow:

Login → Add Course (with name, duration, details) → Course stored in database → Visible to all students → View list of enrolled students → Track Student Progress

DFD level 1


User
registration details
email & password
Registration
user data
student table
faculty table
login
verify credentials
login response
Student
Faculty
course req
enrollment request
lesson request
completion update
status request
course info
lesson info
student data
completion verification
1.0
view course
course data request
course details
course table
insert course data
6.0
add course
2.0
view lesson
lesson data request
lesson details
lesson table
insert lesson data
7.0
add lessons
3.0
enroll course
enrollment data
student table
fetch student info
8.0
view student
fetch progress data
4.0
complete lesson
update status
enrollment table
9.0
verify completion
5.0
course status
fetch status
status details
update/verify status

3.3ACTIVITY DIAGRAM

An activity diagram visually represents the series of actions or flow of control in a system, similar to a flow chart or data flow diagram. Activity diagrams are often used in business process modelling. They can also describe steps in a use case diagram. The activity diagram for the Student module and Faculty module of the Student Course Management System is given below.

3.4GANTT CHART

A Gantt chart is a type of bar chart, devised by Henry Gantt in the 1910s, that illustrates a project schedule. Gantt charts illustrate the start and finish dates of the terminal elements and summary elements of a project. Terminal elements and summary elements comprise the work breakdown structure of the project. Modern Gantt charts also show the dependency (i.e., precedence network) relationships between activities.

HISTORICAL DEVELOPMENT:

The first known tool of this type was developed in 1896 by Karol Adamiecki, who called it a Harmonogram. Adamiecki did not publish his chart until 1931, and only in Polish, which limited both its adoption and recognition of his authorship. The chart is named after Henry Gantt (1861-1919), who designed his chart around the years 1910-1915. One of the first major applications of Gantt charts was by the United States during World War I. In the 1980s, personal computers allowed widespread creation of complex and elaborate Gantt charts.

GANTT CHART BENEFITS:

Clarity:

One of the biggest benefits of a Gantt chart is the tool's ability to boil down multiple tasks and timelines into a single document. Stakeholders throughout an organization can easily understand where teams are in a process while grasping the ways in which independent elements come together toward project completion.

Communication:

Teams can use Gantt charts to replace meetings and enhance other status updates. Simply clarifying chart positions offers an easy, visual method to help team members understand task progress.

Motivation:

Some teams or team members become more effective when faced with a form of external motivation. Gantt charts offer teams the ability to focus work at the front of a task timeline, or at the tail end of a chart segment.

Coordination:

For project managers and resource schedulers, the benefits of a Gantt chart include the ability to sequence events and reduce the potential for overburdening team members. Some project managers even use combinations of charts to break down projects into more manageable sets of tasks.

Time Management:

Most managers regard scheduling as one of the major benefits of Gantt charts. Helping teams understand the overall impact of project delays can foster stronger collaboration while encouraging better task organization.

Flexibility:

The ability to issue new charts as your project evolves lets you react to unexpected changes in project scope or timeline. Offering a realistic view of a project can help team members recover from setbacks or adjust to other changes.

Efficiency:

Another one of the benefits of Gantt charts is the ability for team members to leverage each other's deadlines for maximum efficiency. Visualizing resource usage during projects allows managers to make better use of people, places, and things.

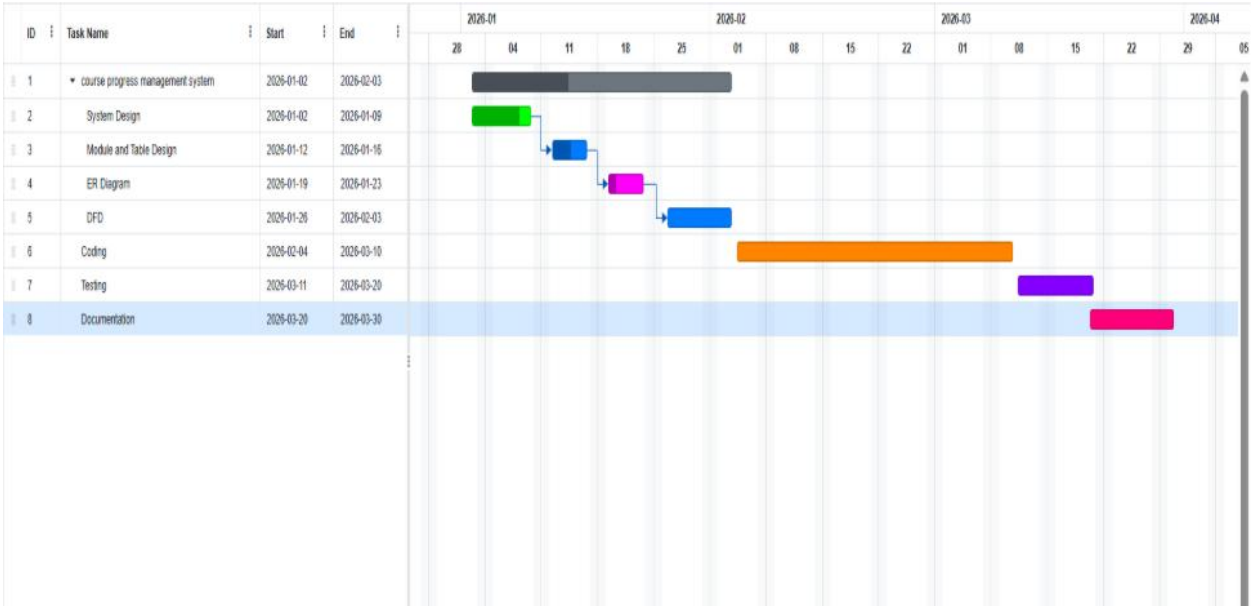
Accountability:

Using Gantt charts during critical projects allows both project managers and participants to track team progress, highlighting both big wins and major failures during professional review periods.

Gantt Chart Importance:

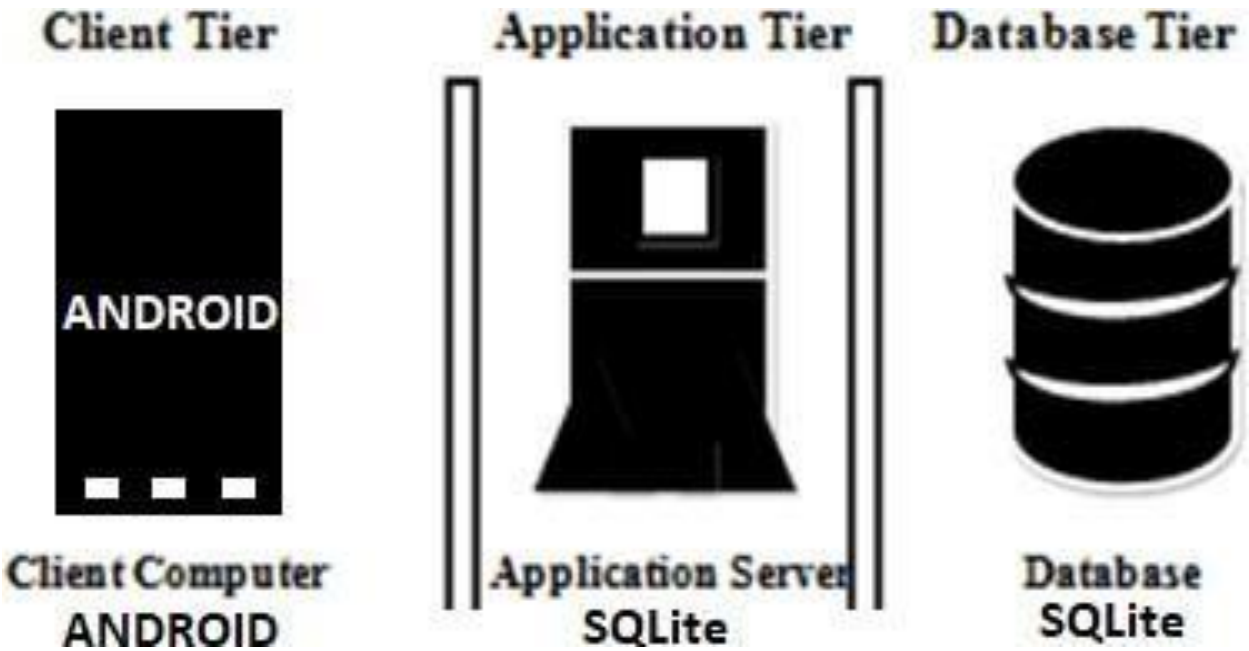
The project's summary and terminal elements, which combine to form the project's internal structure, are shown on the Gantt chart. Many charts will also depict the precedence rankings and dependencies of various tasks within the project. The way to create this chart begins by determining and listing the necessary activities, sketching how the chart should look, listing which items depend on others, and determining the throughput time.

GANTT CHART



3.5ARCHITECTURAL DESIGN

An architectural model (in software) is a rich and rigorous diagram, created using available standards, in which the primary concern is to illustrate a specific set of tradeoffs inherent in the structure and design of a system or ecosystem. The Student Course Management System follows a three-tier architecture: Presentation Layer (Frontend - HTML/CSS/JS), Logic Layer (REST API via Supabase), and Data Layer (PostgreSQL database).

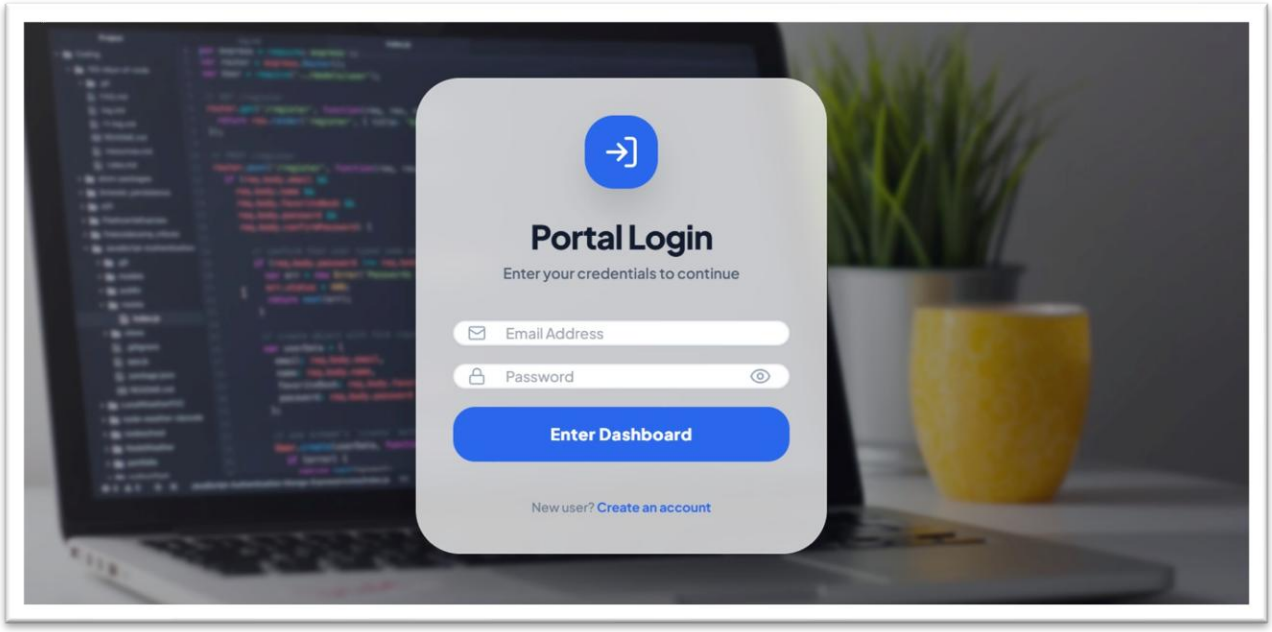


INPUT / OUTPUT DESIGN

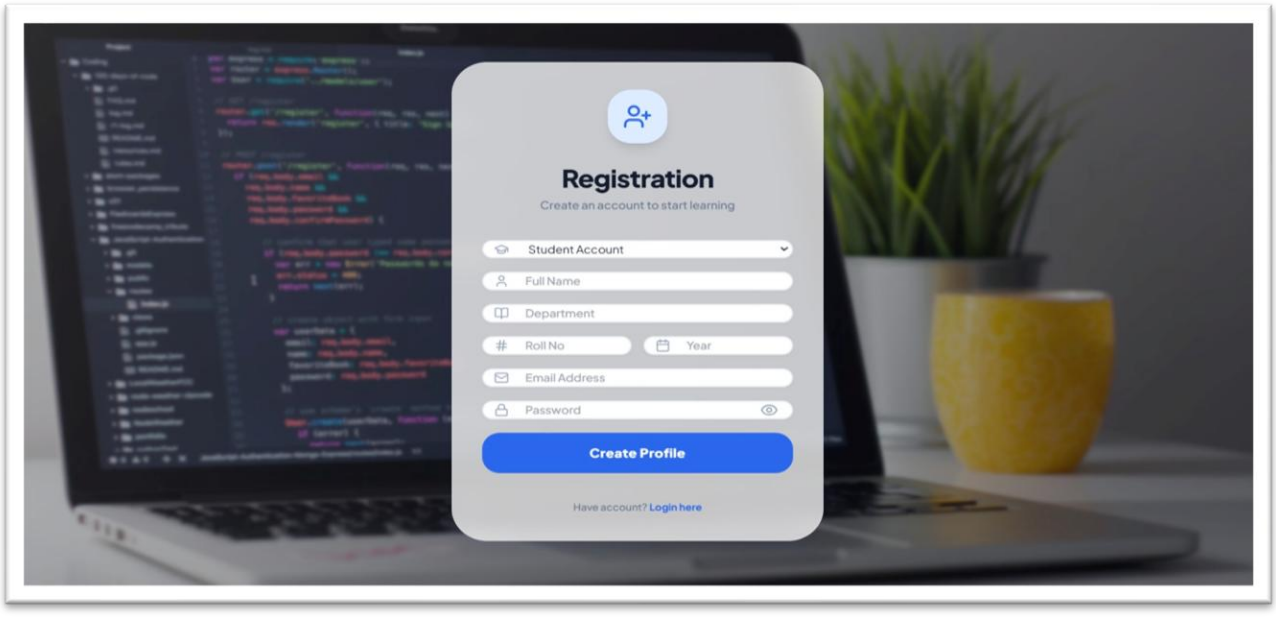
3.6 INPUT / OUTPUT DESIGN

Input Design:

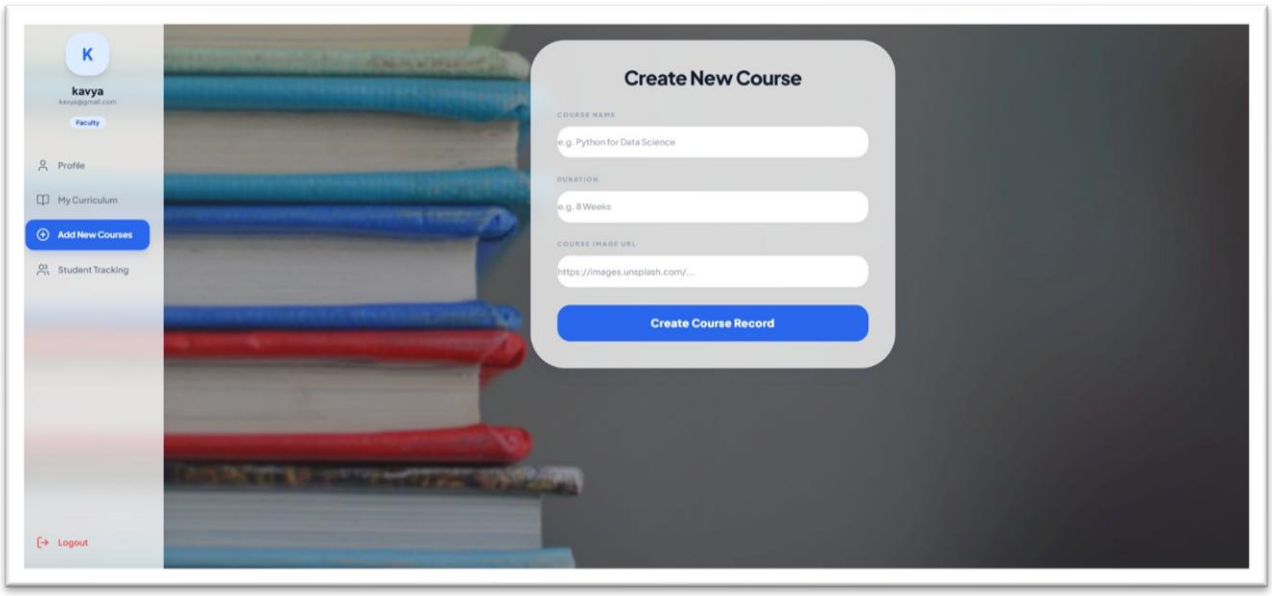
Login form:



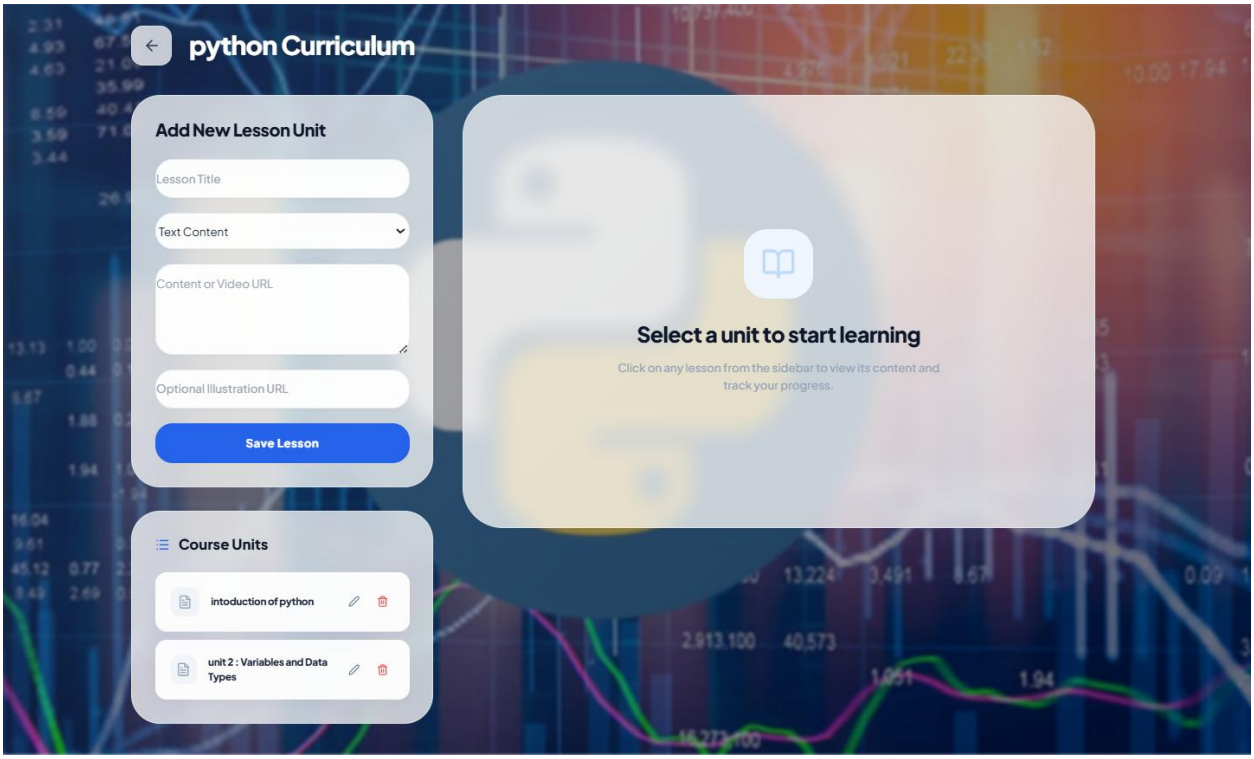
Registration form:



Faculty "Create New Course" Form:



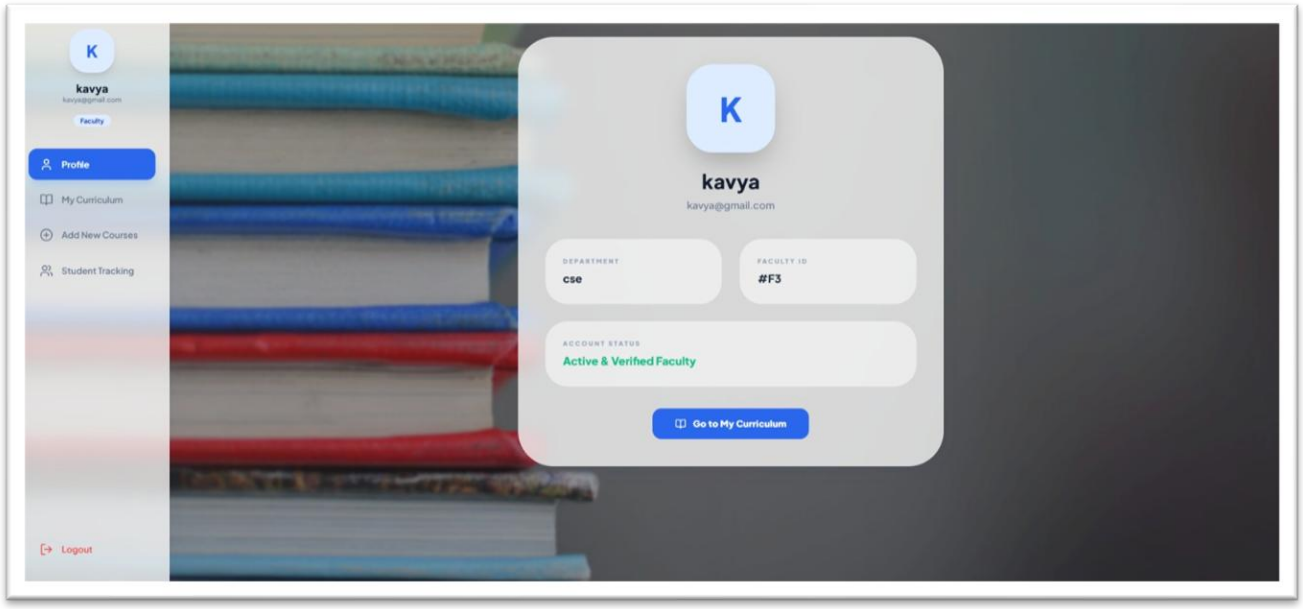
Faculty "Add New Lesson Unit" Form:



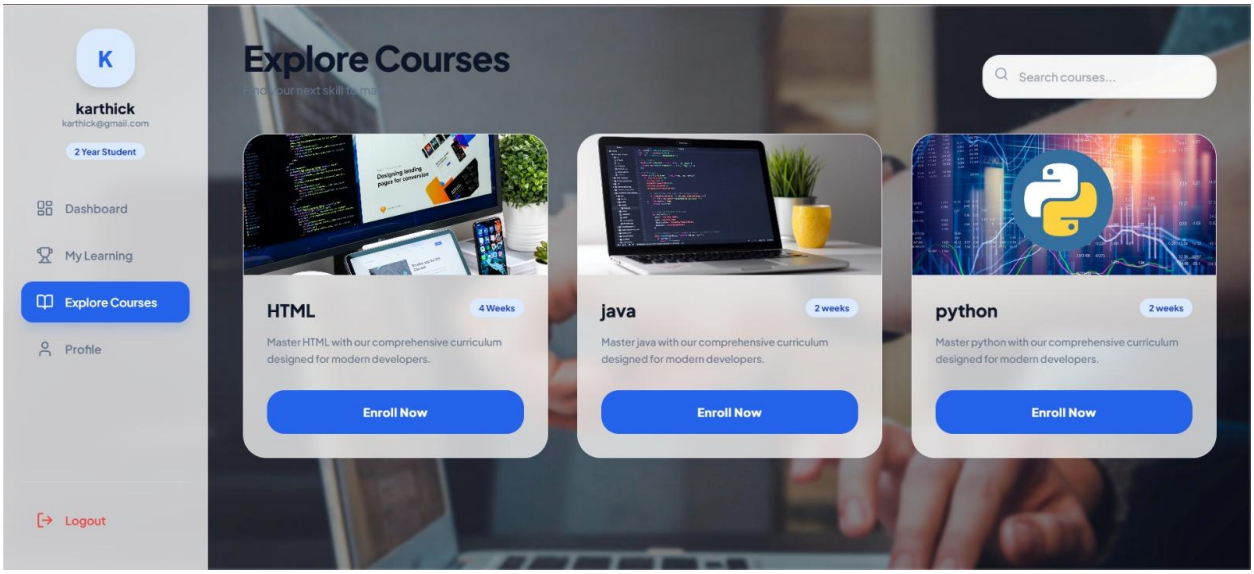
OUTPUT DESIGN

Output Design:

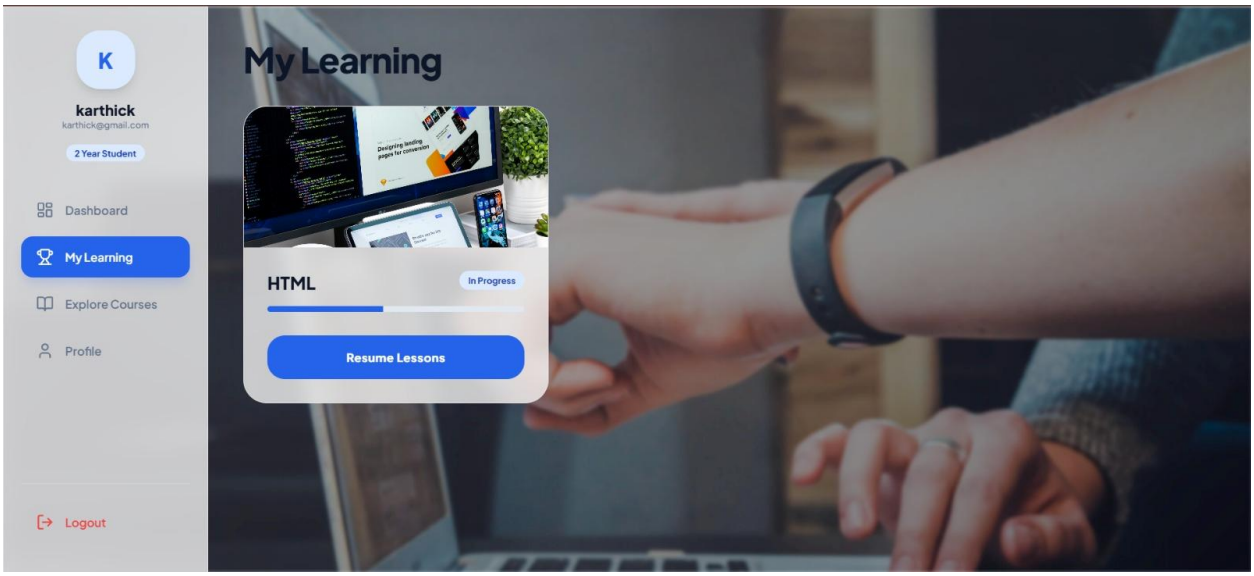
Faculty Profile Display:



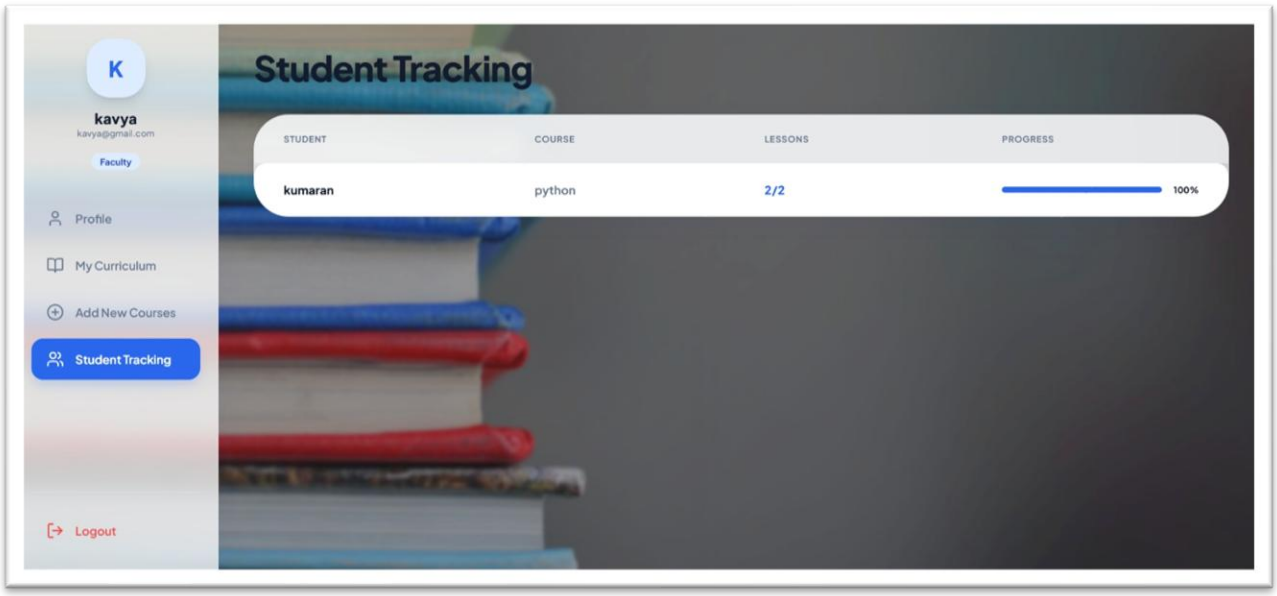
Explore Courses Catalog:



Student "My Learning" Progress View:



Faculty Student Tracking Table:



4.SYSTEM CONFIGURATION

Hardware Requirements

Component	Specification
RAM	Minimum 16 GB
Hard Disk	Minimum 477 B
Processor	12th Gen Intel(R) Core(TM) i5-12450H (2.00 GHz)
Processing Speed	2.00 GHz

Software Requirements

Component	Specification
Front End	HTML5, CSS3, JavaScript (ES6+)
Back End	Supabase (REST API, Authentication)
Tools / IDE	Visual Studio Code (VS Code)
Operating System	Windows 11

5.DETAILS OF SOFTWARE

A development process consists of various phases, each phase ending with a defined output. The phases are performed in an order specified by the process model being followed. The main reason for having a phased process is that it breaks the problem of developing software into successfully performing a set of phases, each handling a different concern of software development.

This ensures that the cost of development is lower than what it would have been if the whole problem were tackled together. A phased development process is central to the software engineering approach for solving the software crisis.

Overview of Front End

The frontend of the Student Course Management System is built using HTML, CSS, and JavaScript. These are the core web technologies used to build the user interface of the application.

HTML (HyperText Markup Language) provides the structure of the web pages, defining elements like forms, tables, buttons, and content areas. CSS (Cascading Style Sheets) is used to style and layout the pages, making the application visually appealing and responsive across different screen sizes and devices. JavaScript provides interactivity and handles user events such as form submissions, button clicks, dynamic page updates, and real-time data display.

The frontend communicates with the Supabase backend through REST API calls using the Supabase JavaScript client library (supabase-js). This allows the application to fetch and store data in real time without requiring a page refresh. The use of these technologies ensures that the application is lightweight, fast, and accessible through any modern web browser without requiring additional plugins or installations.

Visual Studio Code (VS Code) is used as the primary development environment (IDE). It provides features like IntelliSense (code completion), integrated Git support, extensions for HTML/CSS/JS development, and a built-in terminal, which greatly enhance the development productivity and code quality.

Overview of Back End

Supabase is an open-source backend platform built on top of PostgreSQL. It provides a ready-to-use REST API, built-in authentication (user management), and real-time data capabilities. Supabase eliminates the need for custom server-side code for basic CRUD (Create, Read, Update, Delete) operations, making it ideal for rapid web application development.

The Student Course Management System uses Supabase to store and retrieve data about students, faculty, courses, and enrollments in real time without the need to set up and manage a separate server. Supabase provides:

- Auto-generated REST API for all database tables
- Row Level Security (RLS) for data access control
- Built-in authentication and user management
- Real-time subscriptions for live data updates
- Dashboard for easy database management and monitoring

PostgreSQL, the underlying database of Supabase, is a powerful, open-source relational database management system. It provides strong data integrity, support for complex queries, foreign key relationships, and transactions, making it ideal for managing the relational data (students, courses, enrollments) in this system.

About the Platform

The Student Course Management System is a web-based platform accessible through any modern web browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge. It is designed to run on both desktop and laptop computers with an active internet connection.

The application uses Supabase's cloud-hosted PostgreSQL database, which means all data is securely stored in the cloud and accessible from anywhere with an internet connection. The

platform is built to be simple, fast, and reliable, providing an easy-to-use interface for both students and faculty members.

The web-based nature of the platform means there is no need to install any additional software on the user's device. Users simply open their web browser, navigate to the application URL, and log in to access all features. This makes the system highly accessible and maintainable.

The system follows a responsive design approach, ensuring that the interface adapts to different screen sizes. Security is maintained through Supabase's built-in Row Level Security (RLS) policies, ensuring that students can only access their own data and faculty can only manage their own courses and students.

6.TESTING

Testing is a vital part of software development, and it is important to start it as early as possible, and to make testing a part of the process of deciding requirements. To get the most useful perspective on your development project, it is worthwhile devoting some thought to the entire lifecycle including how feedback from users will influence the future of the application.

Testing is part of a lifecycle. The software development lifecycle is one in which you hear of a need, you write some code to fulfil it, and then you check to see whether you have pleased the stakeholders - the users, owners, and other people who have an interest in what the software does. Hopefully they like it, but would also like some additions or changes, so you update or augment your code; and so the cycle continues.

SOFTWARE DEVELOPMENT LIFE CYCLE

Testing is a proxy for the customer. This system "Student Course Management System" is developed using the Incremental Model, where the system is built and tested module by module. Each module (Student, Faculty, Course, Enrollment) is independently developed and tested before integration.

SOFTWARE TESTING TYPES:

1. FUNCTIONAL TESTING

This type of testing ignores the internal parts and focuses on whether the output is as per the requirement or not.

Black Box Testing - Internal system design is not considered in this type of testing. Tests are based on requirements and functionality. For example, testing whether student login correctly authenticates and redirects to the dashboard.

White Box Testing - This testing is based on knowledge of the internal logic of an application's code. Also known as Glass Box Testing. Tests are based on coverage of code statements, branches, paths, and conditions. For example, testing the Supabase query logic for enrollment.

Unit Testing - Testing of individual software components or modules. For this system: testing the login function, the course addition function, the enrollment function, and the progress tracking function independently.

System Testing - The entire system is tested as per the requirements. Black-box type testing that is based on overall requirements specifications, covering all combined parts of the system.

Acceptance Testing - Normally this type of testing is done to verify if the system meets the customer-specified requirements. Students and faculty are the end users who verify that the system fulfills their needs.

Alpha Testing - In-house virtual user environment can be created for this type of testing. Testing is done at the end of development. Minor design changes may be made as a result of such testing.

2. NON-FUNCTIONAL TESTING

Security Testing - Testing how well the system protects against unauthorized internal or external access. Verified that the Supabase database and API are protected using Row Level Security (RLS) policies. Checked that the system is safe from unauthorized data access.

Usability Testing - User-friendliness check. Application flow is tested to verify that new users (students and faculty) can understand the application easily, proper error messages are displayed whenever a user makes a mistake, and the system navigation is intuitive and straightforward.

7 .CONCLUSION AND FUTURE ENHANCEMENT

The Student Course Management System has been successfully developed as a web-based application that simplifies and centralizes the interaction between students and faculty in an academic environment. The system provides an easy-to-use platform where faculty can create and manage courses, and students can browse, enroll in, and track the progress of their enrolled courses.

The system was designed with simplicity and accessibility in mind, ensuring that both students and faculty members can use it with minimal training. By utilizing HTML, CSS, JavaScript, and Supabase/PostgreSQL, the system provides real-time data management and a reliable, scalable backend. The application successfully reduces manual effort, eliminates paperwork, and provides a centralized solution for academic course management.

The system is designed with easy navigation, keeping in mind that the primary users are students and faculty who need a simple and efficient tool for academic course management. The system provides real-time data updates, ensuring that course information and enrollment statuses are always current and accurate. The application provides a clean and organized interface that simplifies the entire process of course management.

Advantages of the System:

- Easy to use interface for both students and faculty
- Real-time data updates using Supabase
- Reduces manual workload and errors
- Centralized and organized course management
- Accessible from any device with a web browser
- Secure data management using PostgreSQL with Row Level Security

FUTURE ENHANCEMENTS

The system can further be enhanced as follows:

- Add a lesson module with course materials, videos, and assignments for students to download and submit.
- Add certificate generation for course completion, allowing students to download certificates automatically.
- Add email and push notifications for enrollment updates, assignment deadlines, and progress reminders.
- Develop a mobile application for iOS and Android platforms for greater accessibility.
- Add an admin panel for overall system management, allowing administrators to manage all users and courses.
- Integrate quiz and assignment submission features with automatic grading.
- Add discussion forums for student-faculty interaction within each course.
- Central server allowing users to access courses from different devices using the same account.
- Keeping track of the dates students have practiced and learned last, to restart lessons on absence for an extended period of time.

8.BIBLIOGRAPHY

Book References:

19. HTML and CSS: Design and Build Websites by Jon Duckett, published by Wiley.
20. JavaScript and JQuery: Interactive Front-End Web Development by Jon Duckett, published by Wiley.
21. PostgreSQL: Up and Running by Regina Obe and Leo Hsu, published by O'Reilly Media.
22. Web Development with HTML5, CSS3 and JavaScript by John Dean, published by Jones & Bartlett Learning.

Web References:

23. https://supabase.com/docs
24. https://www.w3schools.com/html/
25. https://developer.mozilla.org/en-US/docs/Web/JavaScript
26. https://www.postgresql.org/docs/
27. https://www.javatpoint.com/web-development

9.APPENDICES A - TABLE STRUCTURE

Table Name: student

Description: Contains all student details including login credentials

Column Name	Data Type	Constraint
student_id	UUID / INT	PRIMARY KEY
Student_name	TEXT	NOT NULL
Email	TEXT	UNIQUE, NOT NULL
department	TEXT	NOT NULL
year	INT	NOT NULL
Roll_no	varchar	NOT NULL
Password	varchar	NOT NULL
Faculty_id	INT	FOREIGN KEY

Table Name: faculty

Description: Contains all faculty details

Column Name	Data Type	Constraint
faculty_id	UUID / INT	PRIMARY KEY
Faculty_name	TEXT	NOT NULL
email	TEXT	UNIQUE, NOT NULL
department	TEXT	NOT NULL
Password	varchar	NOT NULL

Table Name: course

Description: Contains all course details created by faculty

Column Name	Data Type	Constraint
course_id	UUID / INT	PRIMARY KEY
course_name	TEXT	NOT NULL

Column Name	Data Type	Constraint
duration	TEXT	NOT NULL
faculty_id	UUID / INT	FOREIGN KEY REFERENCES faculty
Total_lesson	INT	NOT NULL

Table Name: enrollment

Description: Contains all enrollment details linking students and courses

Column Name	Data Type	Constraint
enroll_id	UUID / INT	PRIMARY KEY
student_id	UUID / INT	FOREIGN KEY REFERENCES student
course_id	UUID / INT	FOREIGN KEY REFERENCES course
status	TEXT	Not null
Enrolment_date	DATE	NOT NULL

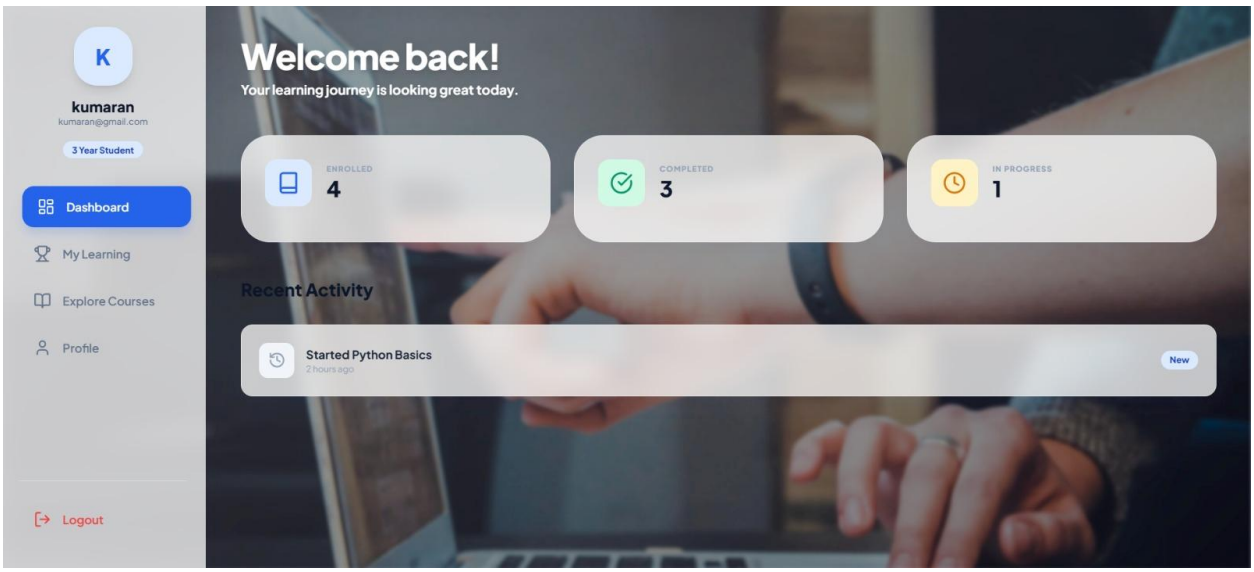
Table Name: lesson
Description: Contains all lesson details and study units linked to specific courses created by faculty.

Column Name	Data Type	Constraint
lesson_id	UUID / INT	PRIMARY KEY
lesson_title	TEXT	NOT NULL
lesson_order	INT	NOT NULL
course_id	UUID / INT	FOREIGN KEY REFERENCES course

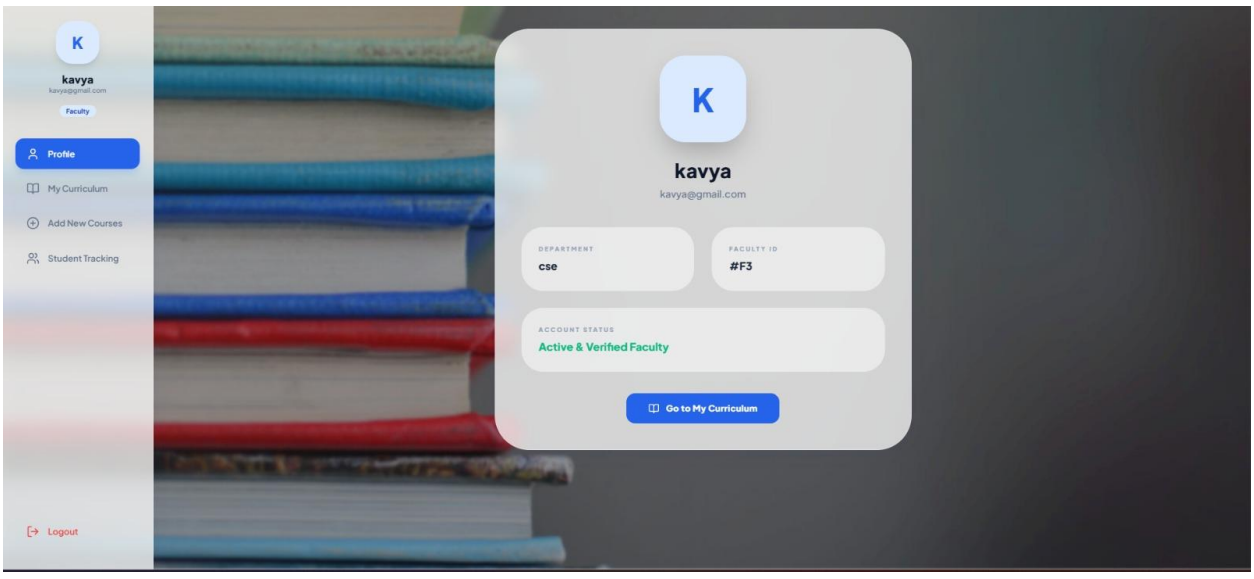
10.APPENDICES B - SCREENSHOTS

(ADD the required screenshots of your application here)

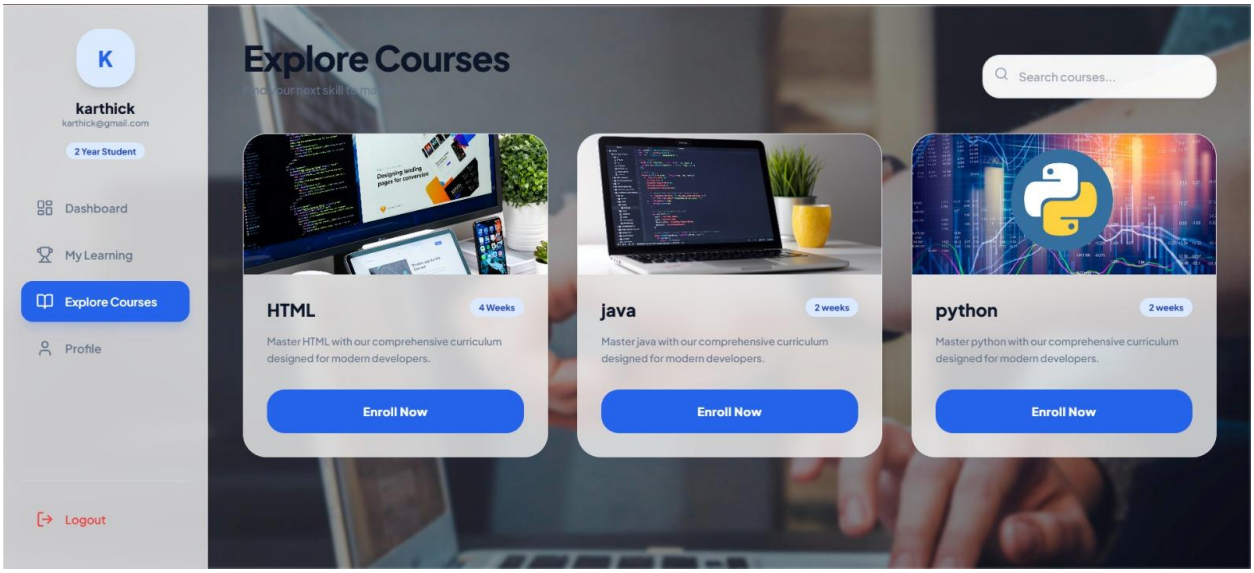
Student Login Page:



Faculty Login Page:

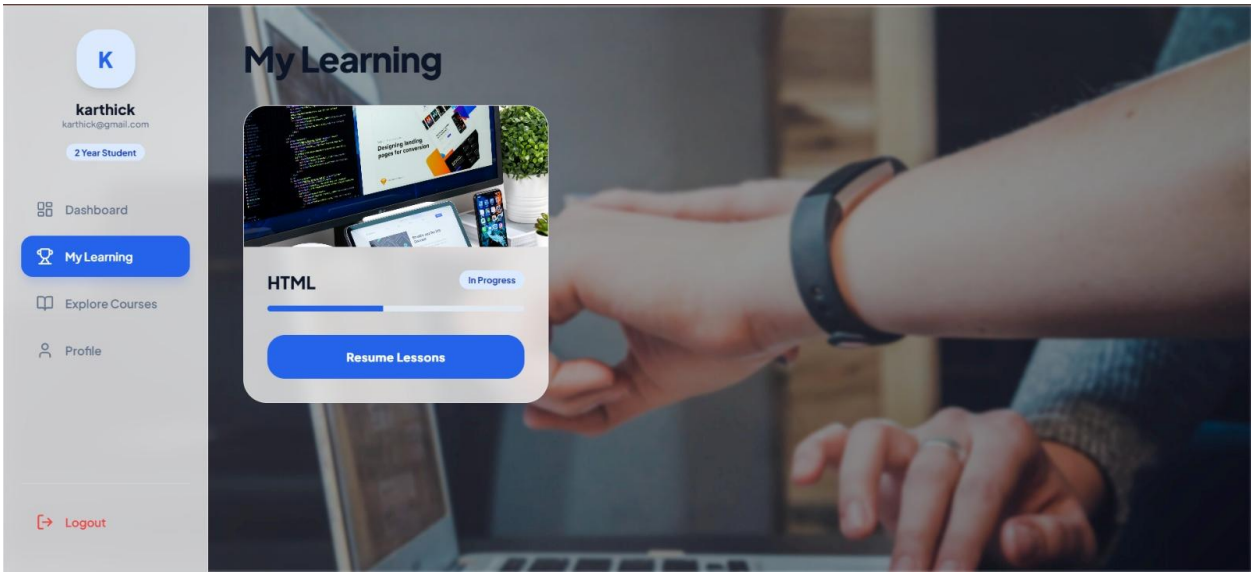


Course List Page (Student View):

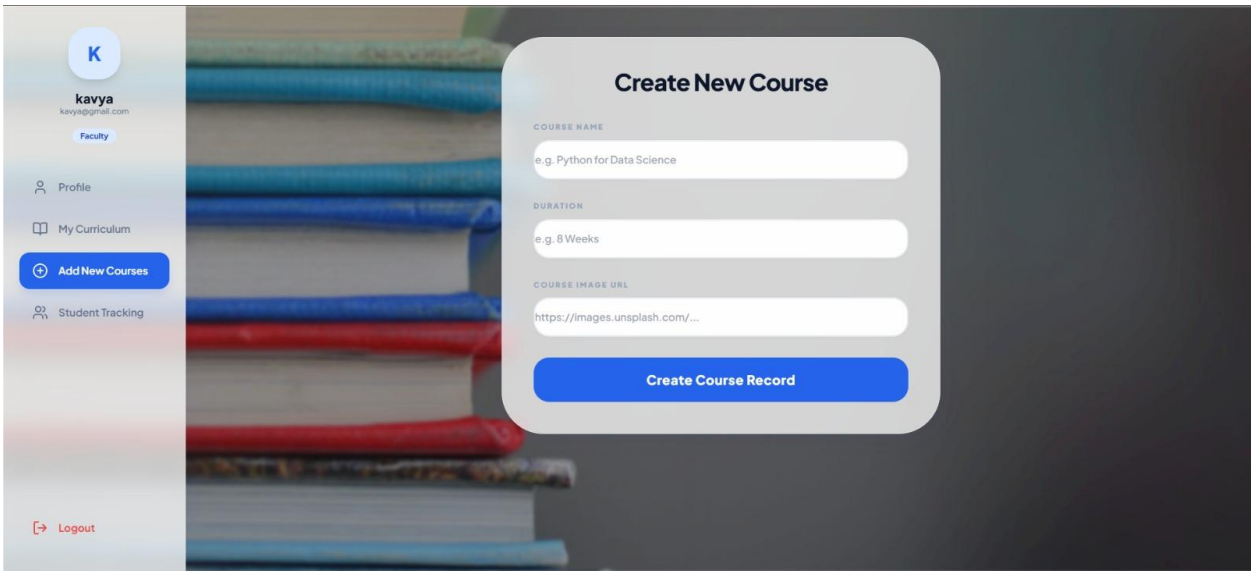


Enroll in Course:

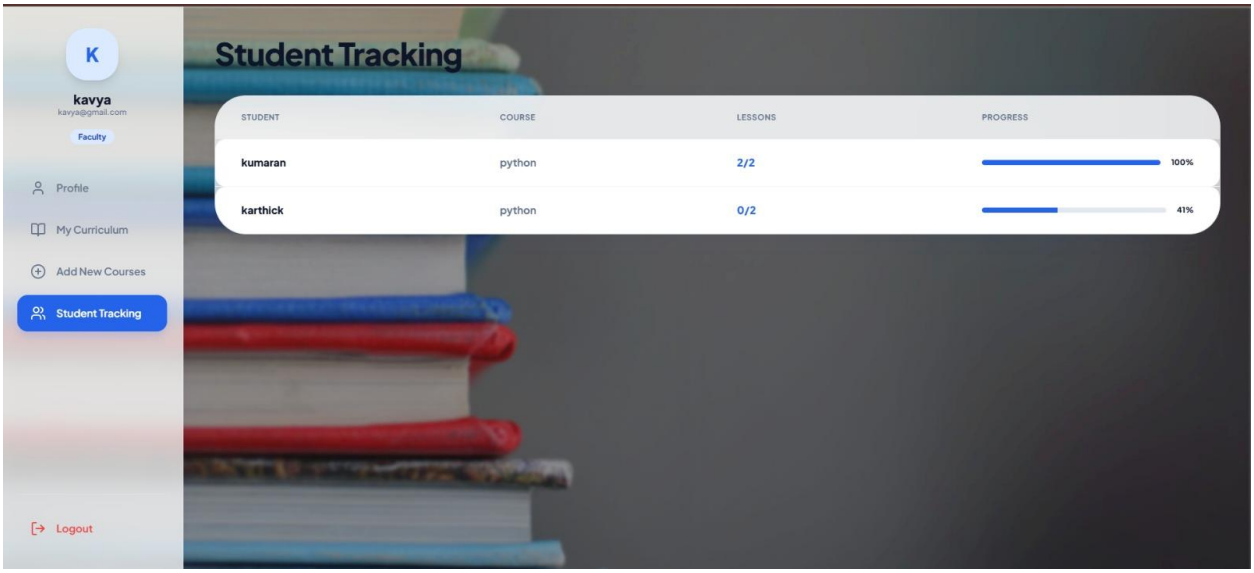
My Courses (Student View):



Add Course (Faculty View):



View Enrolled Students (Faculty View):



11.APPENDICES C - SAMPLE REPORT OF TEST CASES

Test Case ID	Module	Test Description	Input	Expected Output	Actual Output	Status
TC01	Login	Student login with valid credentials	Valid email & password	Redirect to student dashboard	Redirect to student dashboard	Pass
TC02	Login	Faculty login with valid credentials	Valid email & password	Redirect to faculty dashboard	Redirect to faculty dashboard	Pass
TC03	Login	Login with invalid credentials	Wrong email/password	Error message displayed	Error message displayed	Pass
TC04	Course	Faculty adds a new course	Course name, duration	Course saved and visible to students	Course saved and visible	Pass
TC05	Enrollment	Student enrolls in a course	Student clicks Enroll	Course added to My Courses	Course added to My Courses	Pass
TC06	Course List	Student views available courses	Student logs in	All courses displayed	All courses displayed	Pass
TC07	My Courses	Student views enrolled courses	Student logs in	Enrolled courses with status shown	Enrolled courses shown	Pass
TC08	Students List	Faculty views	Faculty logs in	Student list per	Student list shown	Pass

Test Case ID	Module	Test Description	Input	Expected Output	Actual Output	Status
		enrolled students		course shown		
TC09	Validation	Registration with empty fields	Empty form submission	Validation error messages	Validation error messages	Pass
TC10	Data Display	Correct data retrieval from Supabase	View any page	Correct data from database	Correct data displayed	Pass

12.APPENDICES C – SOURCE CODE

1.AUTHENTICATION LOGIC

```
// login.js
const supabaseUrl = 'https://ddxznuiarrmsjkhscvzs.supabase.co';
const supabaseKey =
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImRkeHpud
WlhcnJtc2praHNjdnpzIiwicm9sZSI6ImFub24iLCJpYXQiOjE3Njc5NTQ2MzcsImV4cCI6MjA4
MzUzMDYzN30.ILDZkHznrPRriDuLhPGWfeo_roeyrg6K5lmR7Wwl-Do';
const { createClient } = window.supabase;
// Renamed to sbClient to avoid conflict with the global library object
const sbClient = createClient(supabaseUrl, supabaseKey);

const loginBox = document.getElementById('loginBox');
const registerBox = document.getElementById('registerBox');
const showRegister = document.getElementById('showRegister');
const showLogin = document.getElementById('showLogin');
const regRole = document.getElementById('regRole');
const studentFields = document.getElementById('studentFields');

showRegister.addEventListener('click', () => {
    loginBox.classList.remove('animate-premium-in');
    loginBox.classList.add('animate-premium-out');
    setTimeout(() => {
        loginBox.classList.add('hidden');
        loginBox.classList.remove('animate-premium-out');
        registerBox.classList.remove('animate-premium-out');
        registerBox.classList.remove('hidden');
        registerBox.classList.add('animate-premium-in');
        lucide.createIcons();
    }, 500);
});

showLogin.addEventListener('click', () => {
    registerBox.classList.remove('animate-premium-in');
    registerBox.classList.add('animate-premium-out');
    setTimeout(() => {
        registerBox.classList.add('hidden');
        registerBox.classList.remove('animate-premium-out');
        loginBox.classList.remove('animate-premium-out');
        loginBox.classList.remove('hidden');
        loginBox.classList.add('animate-premium-in');
        lucide.createIcons();
    }, 500);
});
```

```
// Password Toggle
function setupPasswordToggle(inputId, toggleId) {
    const input = document.getElementById(inputId);
    const toggle = document.getElementById(toggleId);

    toggle.addEventListener('click', () => {
        const isPassword = input.type === 'password';
        input.type = isPassword ? 'text' : 'password';
        toggle.setAttribute('data-lucide', isPassword ? 'eye-off' : 'eye');
        lucide.createIcons();
    });
}

setupPasswordToggle('loginPassword', 'toggleLoginPass');
setupPasswordToggle('regPassword', 'toggleRegPass');

regRole.addEventListener('change', (e) => {
    if (e.target.value === 'student') {
        studentFields.classList.remove('hidden');
    } else {
        studentFields.classList.add('hidden');
    }
});

document.getElementById('loginBtn').addEventListener('click', async () => {
    const email = document.getElementById('loginEmail').value;
    const password = document.getElementById('loginPassword').value;

    if (!email || !password) {
        alert("Fields cannot be empty");
        return;
    }

    try {
        const { data: student, error: sError } = await sbClient
            .from("student")
            .select("*")
            .eq("email", email)
            .eq("password", password);

        if (sError) throw sError;

        if (student && student.length > 0) {
            localStorage.setItem("userEmail", email);
            localStorage.setItem("userName", student[0].student_name);
```

```
            localStorage.setItem("userRole", "student");
            window.location.href = "student-dashboard.html";
            return;
        }

        const { data: faculty, error: fError } = await sbClient
            .from("faculty")
            .select("*")
            .eq("email", email)
            .eq("password", password);

        if (fError) throw fError;

        if (faculty && faculty.length > 0) {
            localStorage.setItem("userEmail", email);
            localStorage.setItem("userName", faculty[0].faculty_name);
            localStorage.setItem("userRole", "faculty");
            window.location.href = "faculty-dashboard.html";
            return;
        }

        alert("Invalid Credentials. Please check your email and password.");
    } catch (error) {
        alert("Connection Error: " + error.message);
    }
});

document.getElementById('registerBtn').addEventListener('click', async () => {
    const role = regRole.value;
    const name = document.getElementById('regName').value;
    const email = document.getElementById('regEmail').value;
    const password = document.getElementById('regPassword').value;
    const dept = document.getElementById('regDept').value;

    if (!email || !password || !name || !dept) {
        alert("Please fill all required fields");
        return;
    }

    // GENERATE UNIQUE ID STARTING WITH 26
    const uniqueId = parseInt("26" + Math.floor(1000 + Math.random() * 9000));

    try {
        if (role === 'student') {
            const roll = document.getElementById('regRoll').value;
            const year = document.getElementById('regYear').value;
```

```
            const { error } = await sbClient.from('student').insert([{
                student_id: uniqueId, // Added to fix not-null constraint
                student_name: name,
                email,
                password,
                department: dept,
                roll_no: roll,
                year
            }]);
            if (error) throw error;
        } else {
            const { error } = await sbClient.from('faculty').insert([{
                faculty_id: uniqueId, // Added to fix not-null constraint
                faculty_name: name,
                email,
                password,
                department: dept
            }]);
            if (error) throw error;
        }
        alert(`Registration successful! Your Unique ID is: ${uniqueId}`);
        showLogin.click();
    } catch (error) {
        alert("Registration Error: " + error.message);
    }
});
```

Faculty dashboard logic:

```
// faculty.js
{
const supabaseUrl = 'https://ddxznuiarrmsjkhscvzs.supabase.co';
const supabaseKey =
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImRkeHpud
WlhcnJtc2praHNjdnpzIiwicm9sZSI6ImFub24iLCJpYXQiOjE3Njc5NTQ2MzcsImV4cCI6MjA4
MzUzMDYzN30.ILDZkHznrPRriDuLhPGWfeo_roeyrg6K5lmR7Wwl-Do';
const { createClient } = window.supabase;
const supabase = createClient(supabaseUrl, supabaseKey);

let faculty = null;
let courses = [];
let trackingData = [];
let searchQuery = '';
let editingCourse = null;
```

```
const email = localStorage.getItem("userEmail");

async function init() {
  if (!email) {
    window.location.href = "login.html";
    return;
  }
  await fetchFaculty();
  await fetchCourses();
  await fetchTrackingData();
  switchView('profile');
}

async function fetchFaculty() {
  const { data, error } = await supabase.from("faculty").select("*").eq("email", email);
  if (error || !data.length) {
    window.location.href = "login.html";
    return;
  }
  faculty = data[0];
  updateProfileUI();
}

async function fetchCourses() {
  // Filter courses by faculty_id if possible, or use the mock logic consistently
  const { data } = await supabase.from("course").select("*");
  // We filter locally to ensure "THEIR courses" logic works as requested
  // In a real app, we'd filter in the query: .eq('faculty_id', faculty.faculty_id)
  courses = (data || []).filter(c => {
    // If the course has a faculty_id, use it. Otherwise use the mock logic for now.
    if (c.faculty_id) return c.faculty_id === faculty.faculty_id;
    return c.course_id % 2 === faculty.faculty_id % 2;
  });
}

async function fetchTrackingData() {
  const { data: enrollments } = await supabase.from("enrollment").select("*");
  const { data: students } = await supabase.from("student").select("*");
  const { data: lessons } = await supabase.from("lesson").select("*");

  // Only track students in courses handled by this faculty
  const myCourseIds = new Set(courses.map(c => c.course_id));

  trackingData = (enrollments || [])
    .filter(e => myCourseIds.has(e.course_id))
```

```
        .map(e => {
            const s = students.find(x => x.student_id === e.student_id);
            const c = courses.find(x => x.course_id === e.course_id);
            const courseLessons = lessons.filter(x => x.course_id === e.course_id);
            const progress = e.status === 'Completed' ? 100 : Math.floor(Math.random() * 80) + 10;
            const lessonsCompleted = Math.floor((progress / 100) * courseLessons.length);

            return {
                studentName: s?.student_name || 'Unknown',
                courseName: c?.course_name || 'Unknown',
                lessonsCompleted: `${lessonsCompleted}/${courseLessons.length}`,
                progress: progress
            };
        });
}

function updateProfileUI() {
    const initials = faculty.faculty_name.split(' ').map(n => n[0]).join('').toUpperCase();
    document.getElementById('userAvatar').innerText = initials;
    document.getElementById('userName').innerText = faculty.faculty_name;
    document.getElementById('userEmail').innerText = faculty.email;

    document.getElementById('profileAvatar').innerText = initials;
    document.getElementById('profileName').innerText = faculty.faculty_name;
    document.getElementById('profileEmail').innerText = faculty.email;
    document.getElementById('profileDept').innerText = faculty.department;
    document.getElementById('profileId').innerText = `#F${faculty.faculty_id}`;
}

function renderStats() {
    document.getElementById('statCourses').innerText = courses.length;
    document.getElementById('statStudents').innerText = trackingData.length;
    document.getElementById('statActive').innerText = trackingData.filter(t => t.progress <
100).length;
}

function renderCurriculum() {
    const curriculumArea = document.getElementById('curriculumArea');
    curriculumArea.innerHTML = '';

    const filtered = courses.filter(c => {
        const [name] = c.course_name.split('|||');
        return name.toLowerCase().includes(searchQuery.toLowerCase());
    });

    filtered.forEach(c => {
```

```
        const [name, img] = c.course_name.split('|||');
        const courseImage = img || `https://picsum.photos/seed/${name}/400/200`;

        const card = document.createElement('div');
        card.className = 'glass-panel course-card animate-scale-in';
        card.innerHTML = `
            <div class="h-48 overflow-hidden relative">
                <img src="${courseImage}" class="course-img" alt="Course" referrerPolicy="no-
referrer">
                <button onclick="openEditImage(${c.course_id})" class="absolute top-4 right-4 w-12
h-12 bg-white/90 backdrop-blur rounded-2xl flex items-center justify-center text-blue-600
shadow-xl hover:bg-white transition-all">
                    <i data-lucide="image" class="w-6 h-6"></i>
                </button>
            </div>
            <div class="p-8">
                <div class="flex justify-between items-start mb-4">
                    <h3 class="text-2xl font-black text-slate-900">${name}</h3>
                    <span class="badge badge-blue">ID: ${c.course_id}</span>
                </div>
                <p class="text-slate-500 text-sm mb-8 font-medium">Duration: ${c.duration}</p>
                <button onclick="manageCurriculum(${c.course_id})" class="btn-primary w-
full">Manage Curriculum</button>
            </div>
        `;
        curriculumArea.appendChild(card);
    });
    lucide.createIcons();
}

function renderTracking() {
    const trackingArea = document.getElementById('trackingArea');
    trackingArea.innerHTML = '';

    trackingData.forEach(t => {
        const row = document.createElement('tr');
        row.innerHTML = `
            <td class="px-10 py-6 font-black text-slate-900">${t.studentName}</td>
            <td class="px-10 py-6 text-slate-500 font-bold">${t.courseName.split('|||')[0]}</td>
            <td class="px-10 py-6 font-black text-blue-600">${t.lessonsCompleted}</td>
            <td class="px-10 py-6">
                <div class="flex items-center gap-4">
                    <div class="progress-container flex-1">
                        <div class="progress-bar" style="width: ${t.progress}%"></div>
                    </div>
                    <span class="font-black text-xs text-slate-900">${t.progress}%</span>
```

```
            </div>
          </td>
        `;
        trackingArea.appendChild(row);
    });
}

async function handleAddCourse() {
    const name = document.getElementById('newCourseName').value;
    const duration = document.getElementById('newCourseDuration').value;
    const img = document.getElementById('newCourseImage').value;

    if (!name || !duration) {
        alert("Please fill required fields");
        return;
    }

    const fullCourseName = img ? `${name}|||${img}` : name;
    // Attempt to include faculty_id for better tracking
    const { error } = await supabase.from("course").insert([{
        course_name: fullCourseName,
        duration,
        faculty_id: faculty.faculty_id
    }]);

    if (error) {
        // Fallback if faculty_id column doesn't exist
        const { error: retryError } = await supabase.from("course").insert([{
            course_name: fullCourseName,
            duration
        }]);
        if (retryError) alert(retryError.message);
        else {
            alert("Course Created Successfully!");
            await fetchCourses();
            switchView('curriculum');
        }
    } else {
        alert("Course Created Successfully!");
        await fetchCourses();
        switchView('curriculum');
    }
}

window.openEditImage = function(courseId) {
    editingCourse = courses.find(c => c.course_id === courseId);
```

```
    if (!editingCourse) return;
    const [name, img] = editingCourse.course_name.split('|||');
    document.getElementById('editImageUrl').value = img || '';
    document.getElementById('editPreviewImg').src = img || '';
    document.getElementById('editPreviewContainer').classList.toggle('hidden', !img);
    document.getElementById('editImageModal').classList.remove('hidden');
}

async function handleSaveImage() {
    if (!editingCourse) return;
    const newUrl = document.getElementById('editImageUrl').value;
    const [name] = editingCourse.course_name.split('|||');
    const updatedName = newUrl ? `${name}|||${newUrl}` : name;

    const { error } = await supabase.from("course").update({ course_name: updatedName
}).eq("course_id", editingCourse.course_id);

    if (error) alert(error.message);
    else {
        alert("Image Updated Successfully!");
        document.getElementById('editImageModal').classList.add('hidden');
        await fetchCourses();
        renderCurriculum();
    }
}

window.manageCurriculum = function(courseId) {
    window.location.href = `lesson.html?courseId=${courseId}&faculty=true`;
}

function switchView(view) {
    const sections = document.querySelectorAll('.view-section');
    sections.forEach(v => {
        v.classList.remove('active', 'view-transition');
        v.style.display = 'none';
    });
    document.querySelectorAll('.sidebar .nav-link').forEach(a => a.classList.remove('active'));

    const views = {
        profile: { id: 'profileView', nav: 'navProfile' },
        curriculum: { id: 'curriculumView', nav: 'navCurriculum' },
        addCourse: { id: 'addCourseView', nav: 'navAddCourse' },
        tracking: { id: 'trackingView', nav: 'navTracking' }
    };

    const target = views[view];
```

```
    const targetEl = document.getElementById(target.id);
    targetEl.style.display = 'block';
    targetEl.classList.add('active', 'view-transition');
    document.getElementById(target.nav).classList.add('active');

    if (view === 'curriculum') renderCurriculum();
    if (view === 'tracking') renderTracking();
}

document.getElementById('navProfile').addEventListener('click', () => switchView('profile'));
document.getElementById('navCurriculum').addEventListener('click', () =>
switchView('curriculum'));
document.getElementById('navAddCourse').addEventListener('click', () =>
switchView('addCourse'));
document.getElementById('navTracking').addEventListener('click', () =>
switchView('tracking'));

document.getElementById('courseSearch').addEventListener('input', (e) => {
    searchQuery = e.target.value;
    renderCurriculum();
});

document.getElementById('newCourseImage').addEventListener('input', (e) => {
    const url = e.target.value;
    const preview = document.getElementById('imagePreview');
    const img = document.getElementById('previewImg');
    if (url) { img.src = url; preview.classList.remove('hidden'); }
    else { preview.classList.add('hidden'); }
});

document.getElementById('addCourseBtn').addEventListener('click', handleAddCourse);
document.getElementById('saveImageBtn').addEventListener('click', handleSaveImage);
document.getElementById('closeEditModal').addEventListener('click', () =>
document.getElementById('editImageModal').classList.add('hidden'));

document.getElementById('logoutBtn').addEventListener('click', () => {
    localStorage.clear();
    window.location.href = "login.html";
});

init();
}
```

Student dashboard logic:

```
// student.js
{
const supabaseUrl = 'https://ddxznuiarrmsjkhscvzs.supabase.co';
const supabaseKey =
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImRkeHpud
WlhcnJtc2praHNjdnpzIiwicm9sZSI6ImFub24iLCJpYXQiOjE3Njc5NTQ2MzcsImV4cCI6MjA4
MzUzMDYzN30.ILDZkHznrPRriDuLhPGWfeo_roeyrg6K5lmR7Wwl-Do';
const { createClient } = window.supabase;
const supabase = createClient(supabaseUrl, supabaseKey);

let student = null;
let courses = [];
let myEnrollments = [];
let searchQuery = '';

const email = localStorage.getItem("userEmail");

async function init() {
  if (!email) {
    window.location.href = "login.html";
    return;
  }
  await fetchStudent();
  await fetchCourses();
  renderStats();
  renderCatalog();
}

async function fetchStudent() {
  const { data, error } = await supabase.from("student").select("*").eq("email", email);
  if (error || !data.length) {
    window.location.href = "login.html";
    return;
  }
  student = data[0];
  updateProfileUI();
  await fetchEnrollments(student.student_id);
}

async function fetchCourses() {
  const { data } = await supabase.from("course").select("*");
  courses = data || [];
}

async function fetchEnrollments(studentId) {
```

```
  const { data: enrolls } = await supabase.from("enrollment").select("*").eq("student_id",
studentId);
  myEnrollments = enrolls || [];
}

function updateProfileUI() {
  const initials = student.student_name.split(' ').map(n => n[0]).join('').toUpperCase();
  document.getElementById('userName').innerText = student.student_name;
  document.getElementById('userEmail').innerText = student.email;
  document.getElementById('userBadge').innerText = `${student.year} Year Student`;
  document.getElementById('userAvatar').innerText = initials;

  document.getElementById('profileName').innerText = student.student_name;
  document.getElementById('profileEmail').innerText = student.email;
  document.getElementById('profileDept').innerText = student.department;
  document.getElementById('profileRoll').innerText = student.roll_number;
  document.getElementById('profileYear').innerText = student.year;
  document.getElementById('profileAvatar').innerText = initials;
}

function renderStats() {
  document.getElementById('statEnrolled').innerText = myEnrollments.length;
  document.getElementById('statCompleted').innerText = myEnrollments.filter(e => e.status
=== 'Completed').length;
  document.getElementById('statProgress').innerText = myEnrollments.filter(e => e.status ===
'In Progress').length;
}

function getCourseImage(courseName, customImg) {
  if (customImg) return customImg;
  const name = courseName.toLowerCase();
  if (name.includes('python')) return 'https://images.unsplash.com/photo-1526374965328-
7f61d4dc18c5?q=80&w=2070&auto=format&fit=crop';
  if (name.includes('java')) return 'https://images.unsplash.com/photo-1517694712202-
14dd9538aa97?q=80&w=2070&auto=format&fit=crop';
  if (name.includes('html') || name.includes('web')) return 'https://images.unsplash.com/photo-
1547658719-da2b51169166?q=80&w=2064&auto=format&fit=crop';
  if (name.includes('react')) return 'https://images.unsplash.com/photo-1633356122544-
f134324a6cee?q=80&w=2070&auto=format&fit=crop';
  if (name.includes('data')) return 'https://images.unsplash.com/photo-1551288049-
bbbda536339a?q=80&w=2070&auto=format&fit=crop';
  return `https://picsum.photos/seed/${courseName}/400/200`;
}

function renderCatalog() {
  const catalogArea = document.getElementById('catalogArea');
```

```
    catalogArea.innerHTML = '';

    const filtered = courses.filter(c => {
        const [name] = c.course_name.split('|||');
        return name.toLowerCase().includes(searchQuery.toLowerCase());
    });

    filtered.forEach(c => {
        const [name, img] = c.course_name.split('|||');
        const courseImage = getCourseImage(name, img);
        const isEnrolled = myEnrollments.some(e => e.course_id === c.course_id);

        const card = document.createElement('div');
        card.className = 'glass-panel course-card animate-scale-in';
        card.innerHTML = `
            <div class="h-48 overflow-hidden">
                <img src="${courseImage}" class="course-img" alt="Course" referrerPolicy="no-
referrer">
            </div>
            <div class="p-8">
                <div class="flex justify-between items-start mb-4">
                    <h3 class="text-2xl font-black text-slate-900">${name}</h3>
                    <span class="badge badge-blue">${c.duration}</span>
                </div>
                <p class="text-slate-500 text-sm mb-8 font-medium leading-relaxed">Master ${name}
with our comprehensive curriculum designed for modern developers.</p>
                ${isEnrolled ? `
                    <button onclick="startStudy(${c.course_id})" class="btn-primary w-full bg-slate-
800 hover:bg-slate-900">Continue Learning</button>
                ` : `
                    <button onclick="enroll(${c.course_id})" class="btn-primary w-full">Enroll
Now</button>
                `}
            </div>
        `;
        catalogArea.appendChild(card);
    });
    lucide.createIcons();
}

function renderMyLearning() {
    const myLearningArea = document.getElementById('myLearningArea');
    myLearningArea.innerHTML = '';

    // Ensure unique courses in display
    const uniqueEnrollments = [];
```

```
    const seenCourses = new Set();

    myEnrollments.forEach(e => {
        if (!seenCourses.has(e.course_id)) {
            seenCourses.add(e.course_id);
            uniqueEnrollments.push(e);
        }
    });

    uniqueEnrollments.forEach(e => {
        const c = courses.find(x => x.course_id === e.course_id);
        if (!c) return;
        const [name, img] = c.course_name.split('|||');
        const courseImage = getCourseImage(name, img);

        const card = document.createElement('div');
        card.className = 'glass-panel course-card animate-scale-in';
        card.innerHTML = `
            <div class="h-48 overflow-hidden">
                <img src="${courseImage}" class="course-img" alt="Course" referrerPolicy="no-
referrer">
            </div>
            <div class="p-8">
                <div class="flex justify-between items-start mb-4">
                    <h3 class="text-2xl font-black text-slate-900">${name}</h3>
                    <span class="badge badge-blue">${e.status}</span>
                </div>
                <div class="progress-container mb-8">
                    <div class="progress-bar" style="width: ${e.status === 'Completed' ? '100%' :
'45%'}"></div>
                </div>
                <button onclick="startStudy(${c.course_id})" class="btn-primary w-full">Resume
Lessons</button>
            </div>
        `;
        myLearningArea.appendChild(card);
    });
}


window.enroll = async function(courseId) {
    if (!student) return;

    // Check for duplicate enrollment
    const isAlreadyEnrolled = myEnrollments.some(e => e.course_id === courseId);
    if (isAlreadyEnrolled) {
```

```
        alert("Already enrolled in this course");
        return;
    }

    const { error } = await supabase.from("enrollment").insert([{
        student_id: student.student_id,
        course_id: courseId,
        status: 'In Progress'
    }]);
    if (error) alert(error.message);
    else {
        alert("Enrolled Successfully!");
        await fetchEnrollments(student.student_id);
        switchView('myLearning');
    }
}

window.startStudy = function(courseId) {
    window.location.href = `lesson.html?courseId=${courseId}`;
}

function switchView(view) {
    const sections = document.querySelectorAll('.view-section');
    sections.forEach(v => {
        v.classList.remove('active', 'view-transition');
        v.style.display = 'none';
    });
    document.querySelectorAll('.sidebar .nav-link').forEach(a => a.classList.remove('active'));

    const views = {
        dashboard: { id: 'dashboardView', nav: 'navDashboard' },
        myLearning: { id: 'myLearningView', nav: 'navMyLearning' },
        courses: { id: 'coursesView', nav: 'navCourses' },
        profile: { id: 'profileView', nav: 'navProfile' }
    };

    const target = views[view];
    const targetEl = document.getElementById(target.id);
    targetEl.style.display = 'block';
    targetEl.classList.add('active', 'view-transition');
    document.getElementById(target.nav).classList.add('active');

    if (view === 'dashboard') renderStats();
    if (view === 'myLearning') renderMyLearning();
    if (view === 'courses') renderCatalog();
}
```

```
document.getElementById('navDashboard').addEventListener('click', () =>
switchView('dashboard'));
document.getElementById('navMyLearning').addEventListener('click', () =>
switchView('myLearning'));
document.getElementById('navCourses').addEventListener('click', () => switchView('courses'));
document.getElementById('navProfile').addEventListener('click', () => switchView('profile'));
document.getElementById('profileBackBtn').addEventListener('click', () =>
switchView('courses'));

document.getElementById('courseSearch').addEventListener('input', (e) => {
   searchQuery = e.target.value;
   renderCatalog();
});

document.getElementById('logoutBtn').addEventListener('click', () => {
   localStorage.clear();
   window.location.href = "login.html";
});

init();
}
```